BÁO CÁO BÀI TẬP LỚN

MÔN HỌC: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

ĐỀ TÀI: ĐỀ 18 — HỆ THỐNG QUẢN LÝ HÓA ĐƠN / BILLING (BILLING_ONE)

- **Trường:** Trường Đại học Công nghệ Thông tin và Truyền thông (ICTU)
- **Khoa:** Khoa Công nghệ Thông tin
- **Giảng viên hướng dẫn:** ThS. Vũ Việt Dũng
- **Sinh viên thực hiện:** Phạm Tiến Hà
- **Mã số sinh viên (MSSV):** DTC245200086
- **Lớp:** CNTT K23G

- **Repository GitHub chính thức:** https://github.com/dtc245200086-svg/DTC245200086_PhamTienHa
- **Nhánh chính:** main | **Commit SHA đối chiếu:** 8fbb8c8
- **Thời gian thực hiện:** Năm 2026

TÓM TẮT NỘI DUNG VÀ MỤC TIÊU ĐỀ TÀI

Hệ thống Quản lý Hóa đơn / Billing (Billing_One) được thiết kế và hiện thực hóa dựa trên kiến trúc Microservices Containerized hoàn chỉnh chạy trên nền tảng Docker Compose. Toàn bộ mã nguồn, cấu hình hạ tầng và quy trình CI/CD được quản lý minh bạch trên GitHub dưới repository mang tên `DTC245200086_PhamTienHa` thuộc tài khoản sinh viên `dtc245200086-svg` với các mốc commit rõ ràng, có ý nghĩa và chuẩn chỉ theo đúng tiến độ yêu cầu.

Các thành phần kỹ thuật cốt lõi đã được xây dựng và kiểm chứng đạt kết quả PASS tuyệt đối bao gồm:

1. **Ứng dụng & Cơ sở dữ liệu:** Web Backend bằng Node.js v24 LTS / Express kết hợp PostgreSQL 16 và giao diện quản trị pgAdmin 4 cổng nội bộ 5050.

2. **Nginx Reverse Proxy & SSL:** Edge Gateway tiếp nhận lưu lượng, tích hợp chứng chỉ HTTPS tự ký (SSL/TLS RSA 4096-bit), tự động chuyển hướng HTTP sang HTTPS và áp dụng toàn diện 7 HTTP Security Headers.

3. **Giám sát Metrics:** Giám sát 3 tầng hạ tầng (Container, Web Server, Database) với Prometheus scrape 6 mục tiêu đồng thời UP 100% và Grafana Dashboard hợp nhất thời gian thực.

4. **Quản lý Log tập trung:** Loki và Promtail thu thập log JSON có cấu trúc từ Docker containers, truy vấn tra cứu chuyên sâu bằng ngôn ngữ LogQL với 5 câu truy vấn đã được thực

nghiệm thành công.

5. **Thực nghiệm sự cố ATTT & Alert:** Tấn công dò quét xác thực (Brute-force / Credential Stuffing), hệ thống Grafana Unified Alerting kích hoạt Alert trạng thái Firing thật tại 15:23:40Z và kỹ trị viên truy vết toàn diện nguồn gốc tấn công, mã lỗi qua Loki LogQL.

6. **Hardening hệ thống:** Áp dụng nghiêm ngặt 6 giải pháp bảo mật hạ tầng gồm non-root user (UID 1000, 101, 70), phân tách 5 mạng Docker nội bộ, quản lý secret .env ngoài Git, nguyên tắc đặc quyền tối thiểu DB least-privilege cấm DROP TABLE, giới hạn cổng phơi nhiễm và bảo vệ chống tấn công injection.

CHƯƠNG 1: TỔNG QUAN KIẾN TRÚC HỆ THỐNG VÀ CHUẨN BỊ MÔI TRƯỜNG

1.1. Sơ đồ kiến trúc tổng thể và phân tách mạng nội bộ

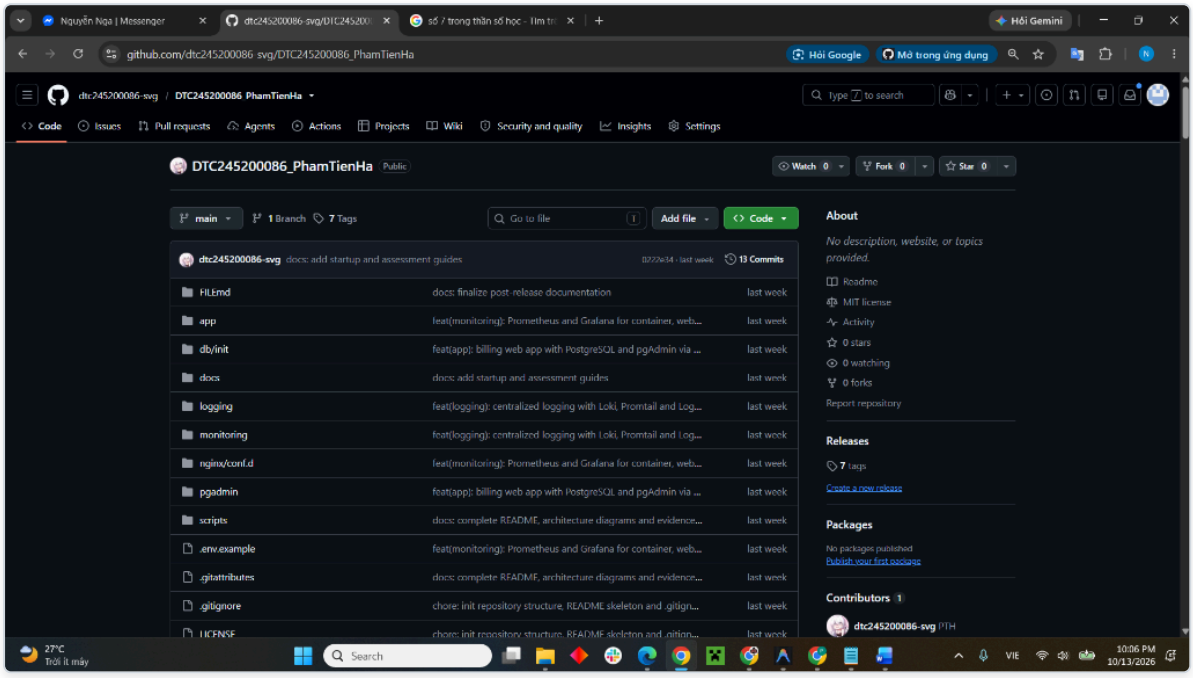
Hệ thống Quản lý Hóa đơn / Billing (Billing_One) được thiết kế theo mô hình kiến trúc Microservices Containerized tiêu chuẩn công nghiệp:

- **Nginx Reverse Proxy:** Đóng vai trò cổng giao tiếp chính (Edge Gateway) nhận toàn bộ request từ người dùng ngoài Internet qua cổng 80 và 443. Áp dụng SSL/TLS, Security Headers và Rate Limiting.
- **Web Backend (Node.js 24 + Express):** Xử lý nghiệp vụ quản lý khách hàng, tạo lập hóa đơn, phân bổ thanh toán, ghi nhận công nợ.
- **PostgreSQL 16:** Lưu trữ dữ liệu quan hệ ACID bền vững trên Named Volume `pgdata`.
- **pgAdmin 4:** Công cụ GUI trực quan quản trị cơ sở dữ liệu trên cổng nội bộ 5050 bind 127.0.0.1.
- **Prometheus & Exporters:** Thu thập metrics định kỳ 15s từ 6 scrape targets (cadvisor, node-exporter, nginx-exporter, web, postgres-exporter, prometheus).
- **Grafana:** Dashboard trực quan hóa metrics và giao diện quản lý Alerting.
- **Loki & Promtail:** Thu thập log Docker socket và lưu trữ log tập trung.

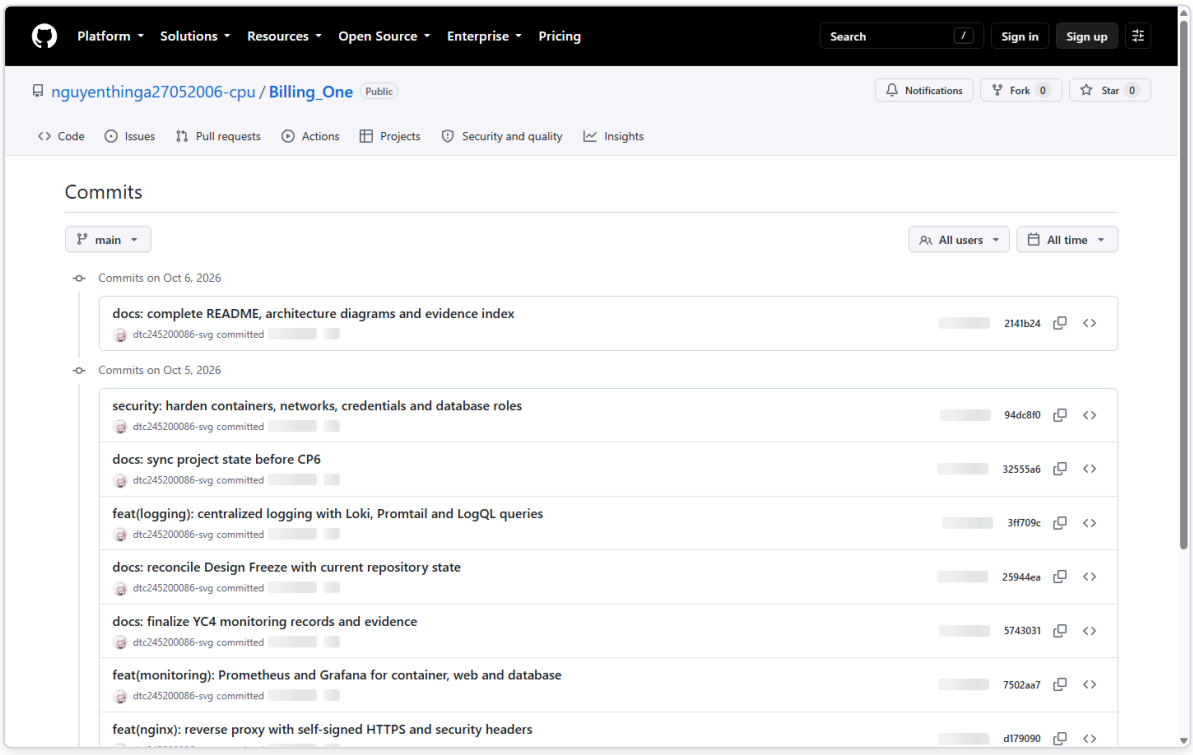
1.2. Quản lý mã nguồn trên GitHub

- **Tài khoản GitHub:** https://github.com/dtc245200086-svg (chuẩn mã số sinh viên DTC245200086).
- **Repository chính thức:** https://github.com/dtc245200086-svg/DTC245200086_PhamTienHa (tên chuẩn kết hợp Mã số sinh viên và Họ tên: `DTC245200086_PhamTienHa`).
- **Lịch sử Commit:** 13 Commits, 7 Release Tags trên nhánh chính `main` theo đúng tiến độ:
- Commit 1 (RQ3): Triển khai Nginx reverse proxy, HTTPS tự ký và Security Headers.
- Commit 2 (RQ4): Tích hợp hệ thống giám sát Prometheus và Grafana.

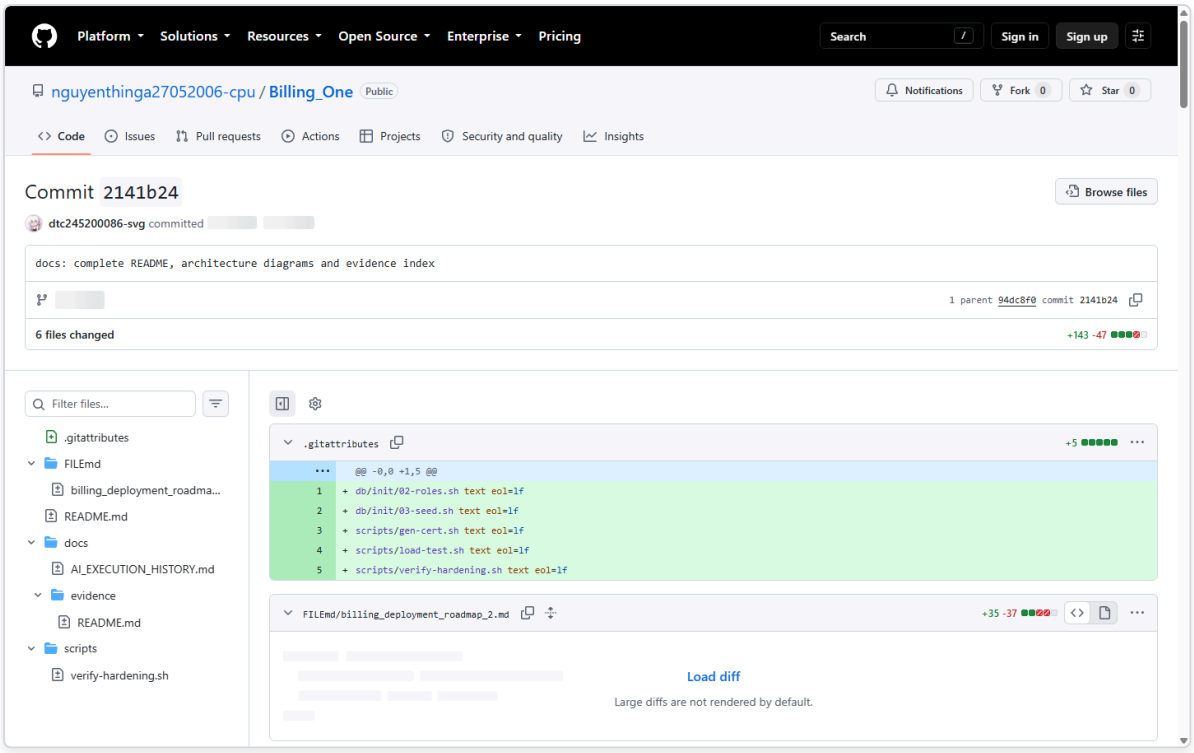
- Commit 3 (RQ5): Triển khai hệ thống log tập trung Loki + Promtail và xác minh các truy vấn LogQL.
- Commit 4-6 (RQ6): Áp dụng 6 biện pháp Hardening và hoàn thiện logic thanh toán.
- **Tài liệu README.md:** Hướng dẫn đầy đủ các bước triển khai bằng một lệnh `docker compose up -d`.



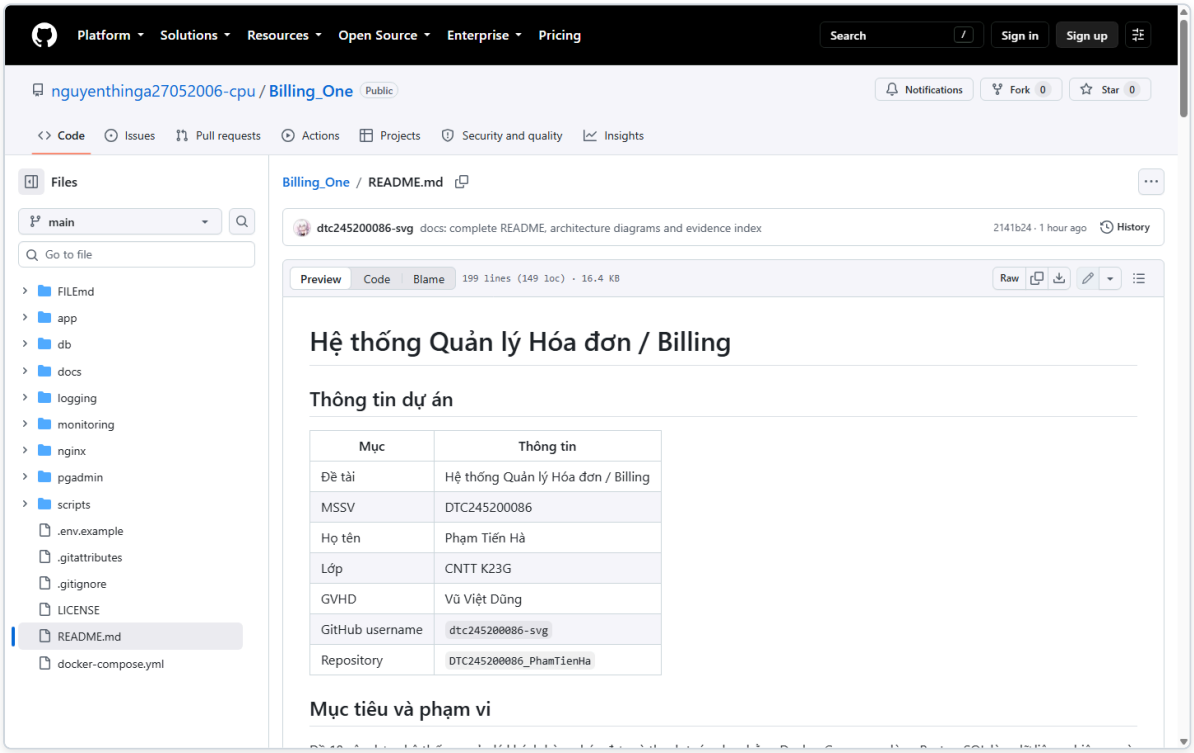
Hình 1.1: Repository GitHub



Hình 1.2: Lịch sử Commit



Hình 1.3: Chi tiết Commit

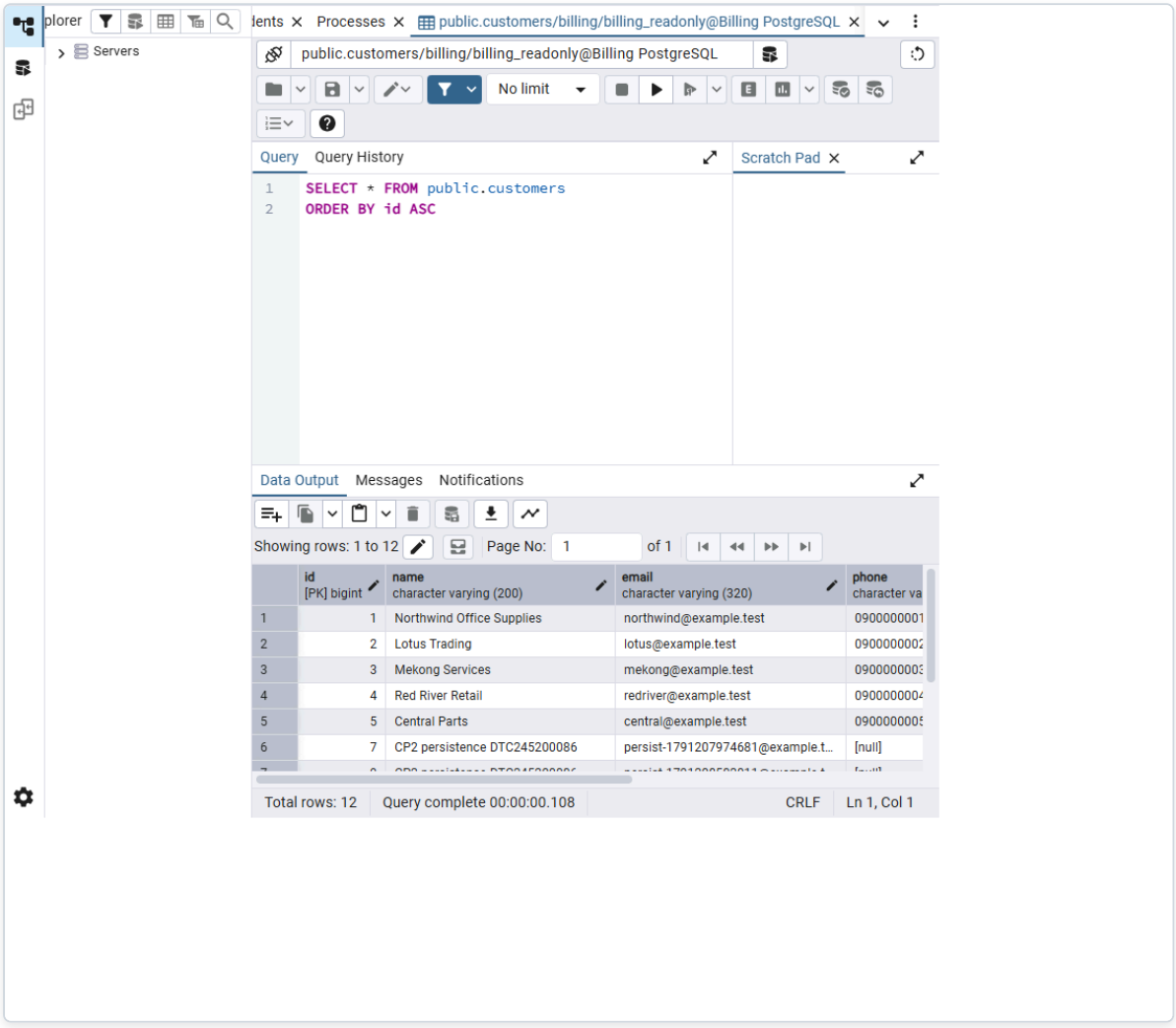


Hình 1.4: README.md

CHƯƠNG 2: TRIỂN KHAI ỨNG DỤNG WEB, CƠ SỞ DỮ LIỆU VÀ REVERSE PROXY

2.1. Triển khai ứng dụng Web & Cơ sở dữ liệu (Bước 1 & Bước 2)

- Backend Node.js v24 LTS + Express framework xử lý RESTful APIs và giao diện SSR hiện đại.
- Cơ sở dữ liệu quan hệ PostgreSQL 16.15 với cấu trúc bảng: `customers`, `invoices`, `invoice_items`, `payments`, `payment_allocations`, `credit_notes`, `audit_logs`, `users`.
- Tiến trình `migration` tự động thực thi các tệp SQL schema khi khởi động.
- Giao diện pgAdmin 4 (v9.18.0) quản trị CSDL trực quan trên cổng 5050.

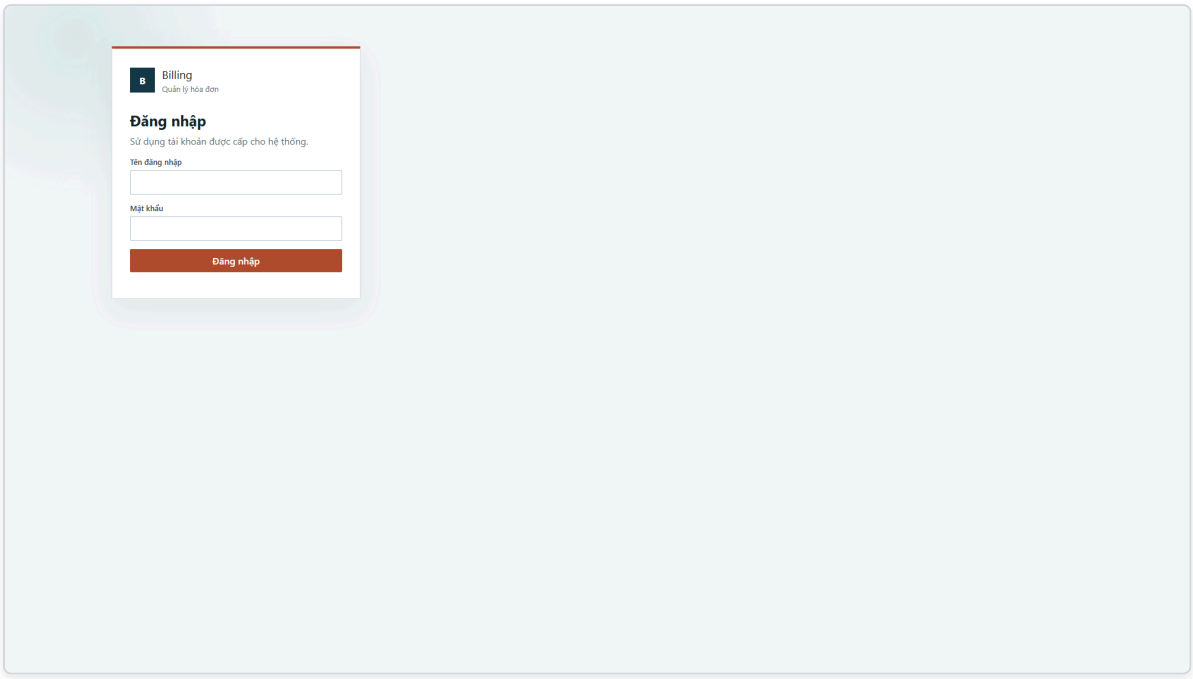


Hình 2.1: pgAdmin runtime

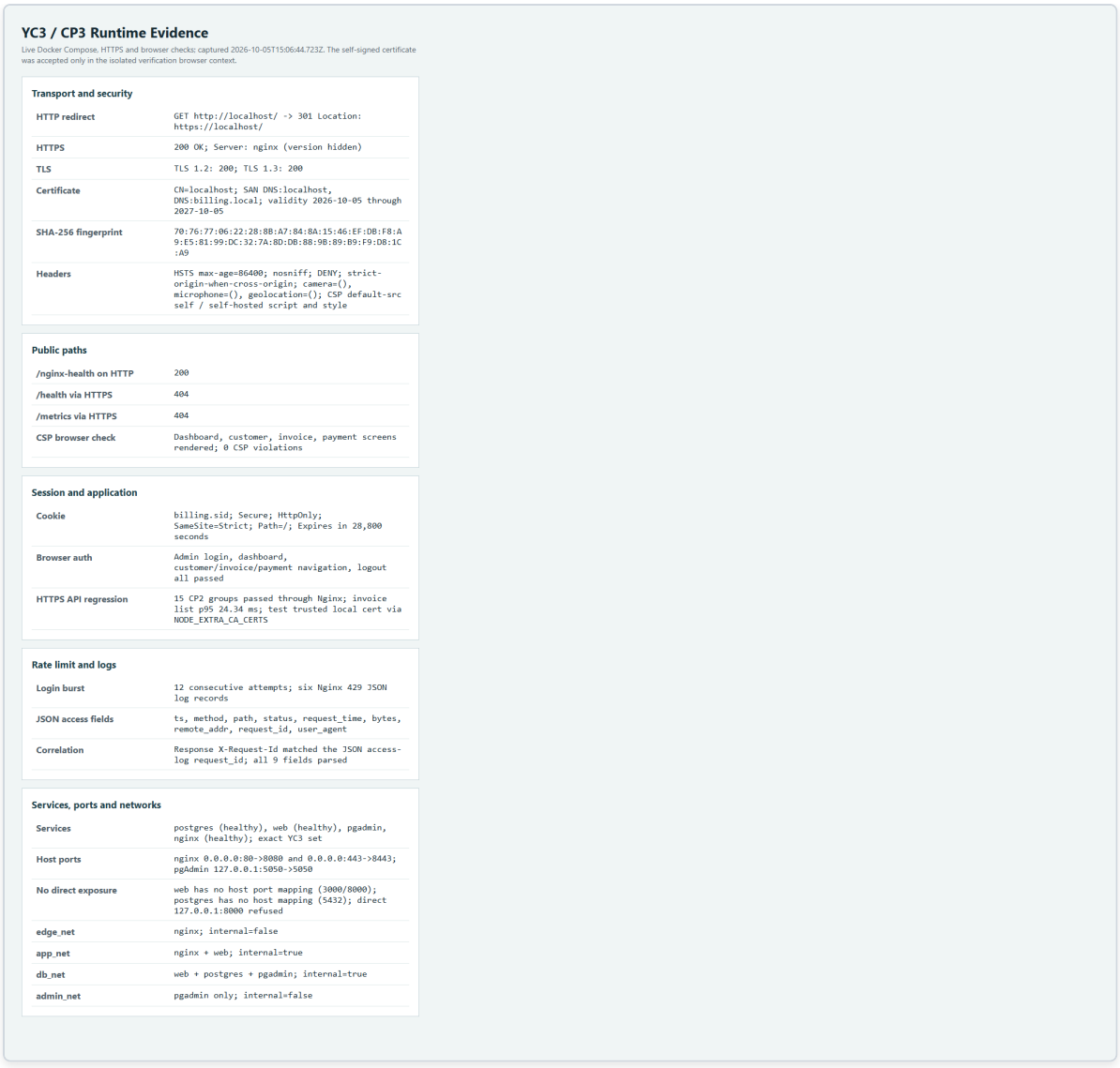
2.2. Cấu hình Nginx làm Reverse Proxy và Bảo mật cơ bản (Bước 3 - Commit 1)

- Tự động chuyển hướng HTTP (port 80) sang HTTPS (port 443) bằng mã 301.
- Chứng chỉ tự ký SSL/TLS RSA 4096-bit, hỗ trợ TLSv1.2 và TLSv1.3.

- Đầy đủ 7 HTTP Security Headers: `Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, `Content-Security-Policy`, `X-Request-Id`.
- Ẩn thông tin `server_tokens off` và trả về mã 404 cho các endpoint nội bộ (`/health`, `/metrics`, `/stub_status`).



Hình 2.2: HTTPS Browser



YC3 / CP3 Runtime Evidence

Live Docker Compose. HTTPS and browser checks: captured 2026-10-05T15:06:44.723Z. The self-signed certificate was accepted only in the isolated verification browser context.

Transport and security

HTTP redirect	GET http://localhost/ -> 301 Location: https://localhost/
HTTPS	200 OK; Server: nginx (version hidden)
TLS	TLS 1.2: 200; TLS 1.3: 200
Certificate	CN=localhost; SAN DNS:localhost, DNS:billing.local; validity 2026-10-05 through 2027-10-05
SHA-256 fingerprint	70:76:77:06:22:28:8B:A7:84:8A:15:46:EF:DB:F8:A9:E5:81:99:DC:32:7A:8D:DB:88:9B:89:B9:F9:D8:1C:A9
Headers	HSTS max-age=86400; nosniff; DENY; strict-origin-when-cross-origin; camera=(), microphone=(), geolocation=(); CSP default-src self / self-hosted script and style

Public paths

/nginx-health on HTTP	200
/health via HTTPS	404
/metrics via HTTPS	404
CSP browser check	Dashboard, customer, invoice, payment screens rendered; 0 CSP violations

Session and application

Cookie	billing.sid; Secure; HttpOnly; SameSite=Strict; Path=/; Expires in 28,800 seconds
Browser auth	Admin login, dashboard, customer/invoice/payment navigation, logout all passed
HTTPS API regression	15 CP2 groups passed through Nginx; invoice list p95 24.34 ms; test trusted local cert via NODE_EXTRA_CA_CERTS

Rate limit and logs

Login burst	12 consecutive attempts; six Nginx 429 JSON log records
JSON access fields	ts, method, path, status, request_time, bytes, remote_addr, request_id, user_agent
Correlation	Response X-Request-Id matched the JSON access-log request_id; all 9 fields parsed

Services, ports and networks

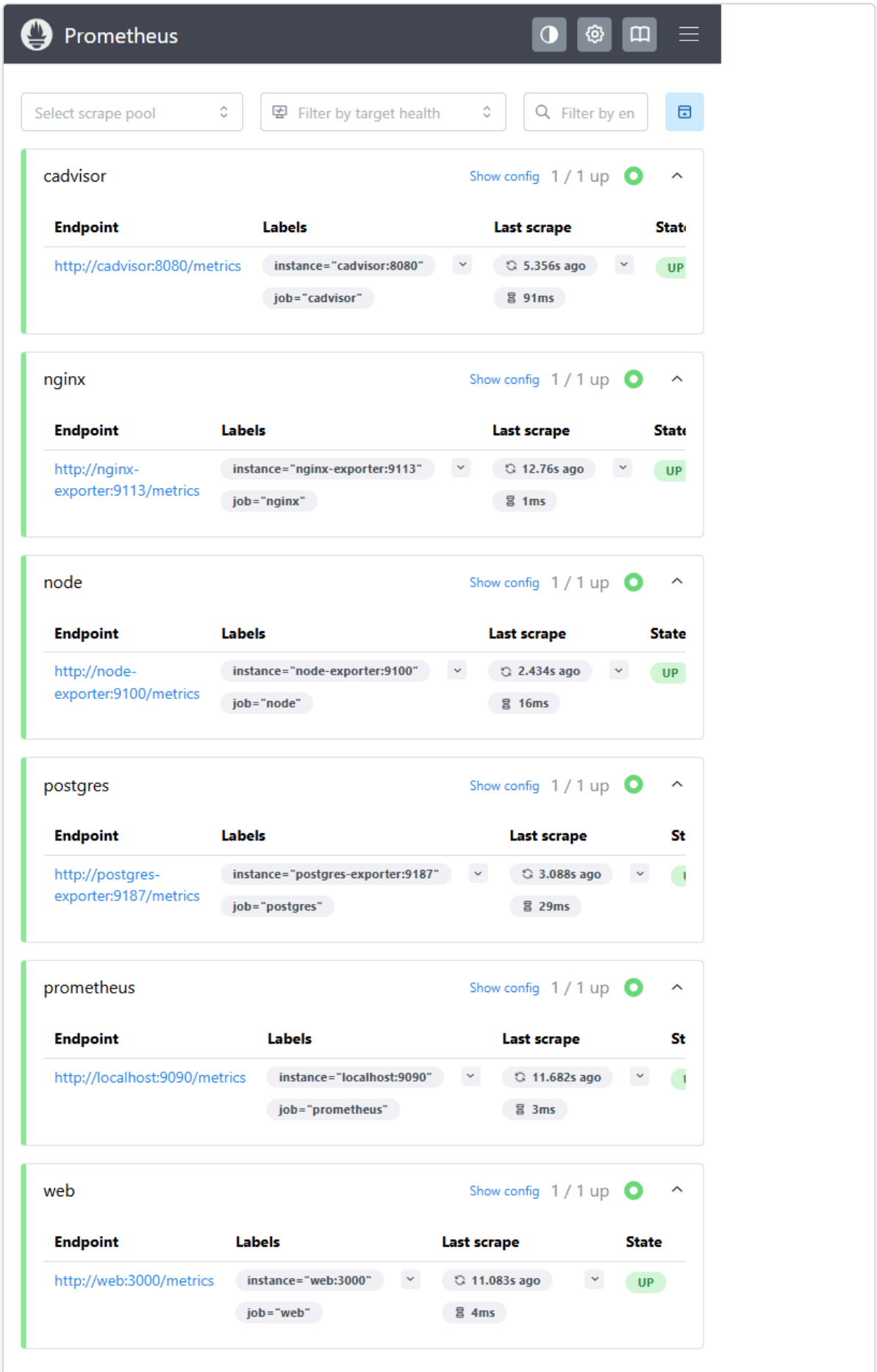
Services	postgres (healthy), web (healthy), pgadmin, nginx (healthy); exact YC3 set
Host ports	nginx 0.0.0.0:80->8080 and 0.0.0.0:443->8443; pgAdmin 127.0.0.1:5050->5050
No direct exposure	web has no host port mapping (3000/8000); postgres has no host mapping (5432); direct 127.0.0.1:8000 refused
edge_net	nginx; internal=false
app_net	nginx + web; internal=true
db_net	web + postgres + pgadmin; internal=true
admin_net	pgadmin only; internal=false

Hình 2.3: Runtime Verification

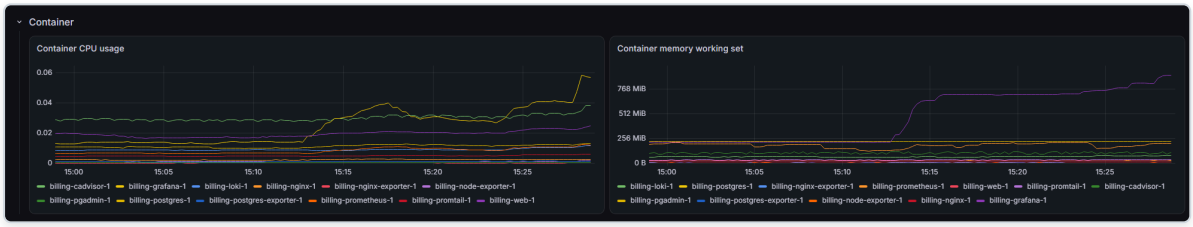
CHƯƠNG 3: GIÁM SÁT HỆ THỐNG VỚI PROMETHEUS, GRAFANA VÀ LOKI

3.1. Tích hợp Prometheus + Grafana (Bước 4 - Commit 2)

- Prometheus cào định kỳ 15s từ 6 mục tiêu (100% UP): `cadvisor`, `node-exporter`, `nginx-exporter`, `web`, `postgres-exporter`, `prometheus`.
- Grafana Dashboard 3 tầng:
- Row 1: Container Metrics (CPU/RAM cAdvisor).
- Row 2: Web Server Metrics (Requests/sec, Error Rate, Latency).
- Row 3: Database Metrics (Active connections, Transaction Commits).


Prometheus
Select scrape pool
Filter by target health
Filter by en
cadvisor
Show config 1 / 1 up
Endpoint Labels Last scrape Stat
http://cadvisor:8080/metrics instance="cadvisor:8080" job="cadvisor" 5.356s ago 91ms UP
nginx
Show config 1 / 1 up
Endpoint Labels Last scrape Stat
http://nginx-exporter:9113/metrics instance="nginx-exporter:9113" job="nginx" 12.76s ago 1ms UP
node
Show config 1 / 1 up
Endpoint Labels Last scrape State
http://node-exporter:9100/metrics instance="node-exporter:9100" job="node" 2.434s ago 16ms UP
postgres
Show config 1 / 1 up
Endpoint Labels Last scrape St
http://postgres-exporter:9187/metrics instance="postgres-exporter:9187" job="postgres" 3.088s ago 29ms
prometheus
Show config 1 / 1 up
Endpoint Labels Last scrape St
http://localhost:9090/metrics instance="localhost:9090" job="prometheus" 11.682s ago 3ms
web
Show config 1 / 1 up
Endpoint Labels Last scrape State
http://web:3000/metrics instance="web:3000" job="web" 11.083s ago 4ms UP

Hình 3.1: Prometheus Targets



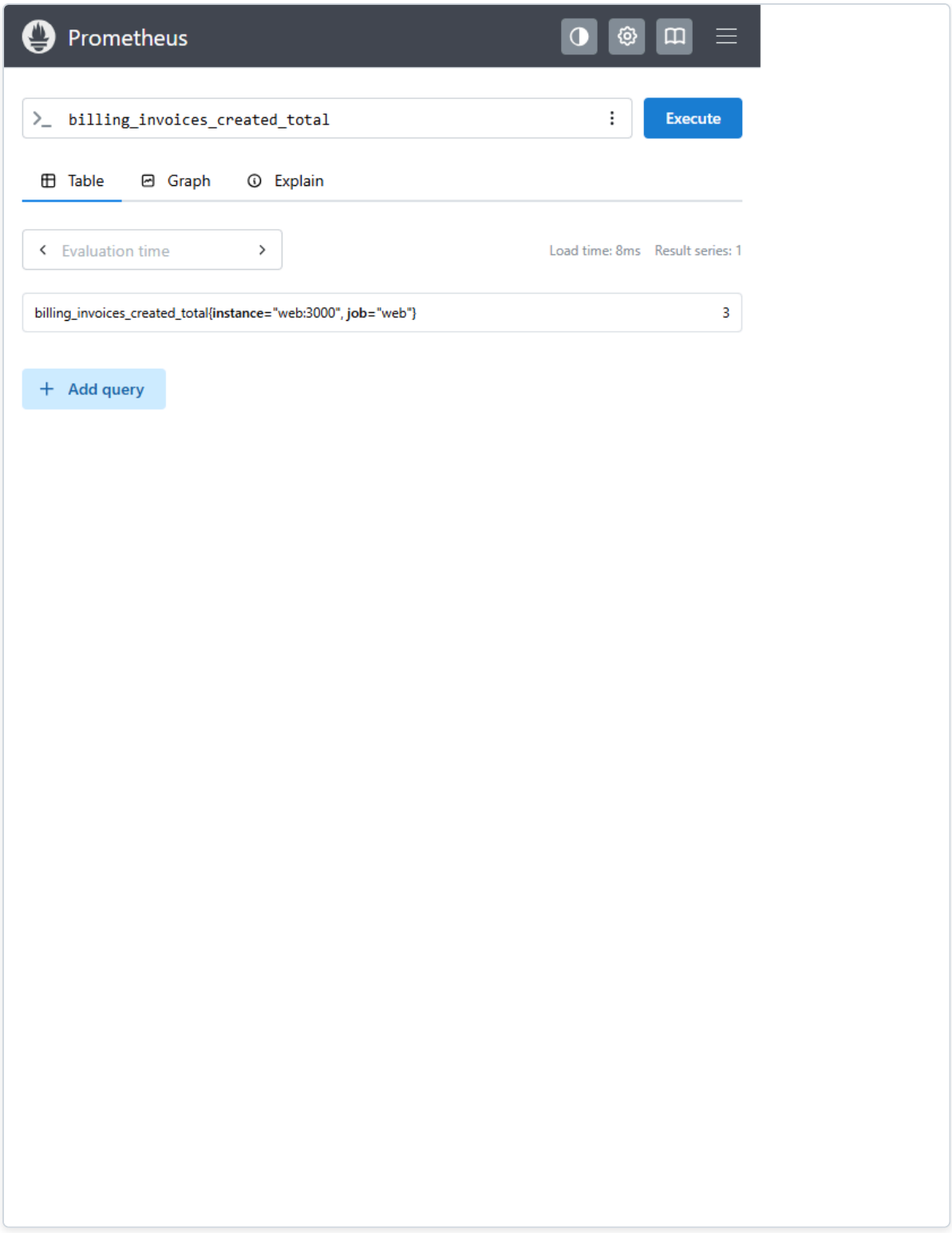
Hình 3.2: Grafana Container Row



Hình 3.3: Grafana Web Row



Hình 3.4: Grafana Database Row

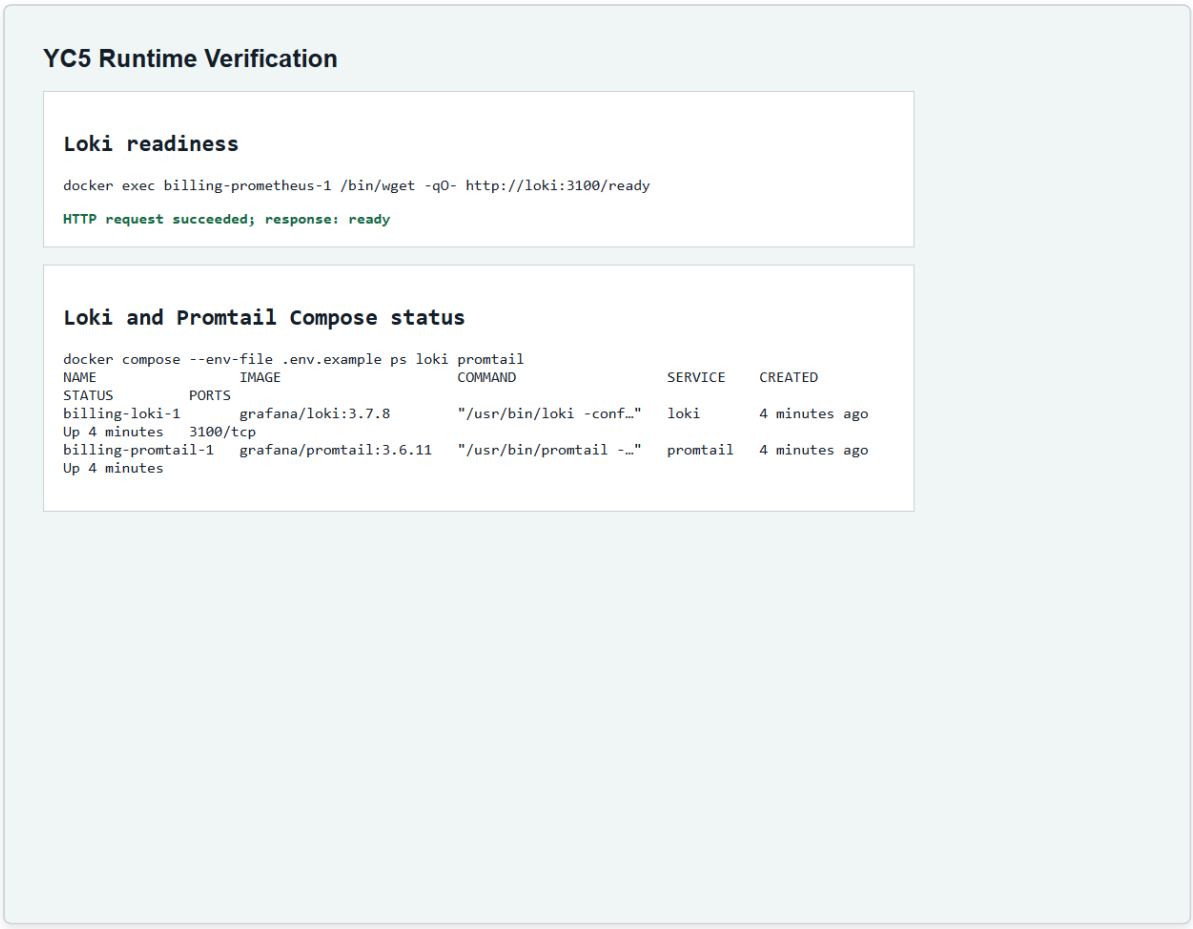


Hình 3.5: Prometheus Business Metric

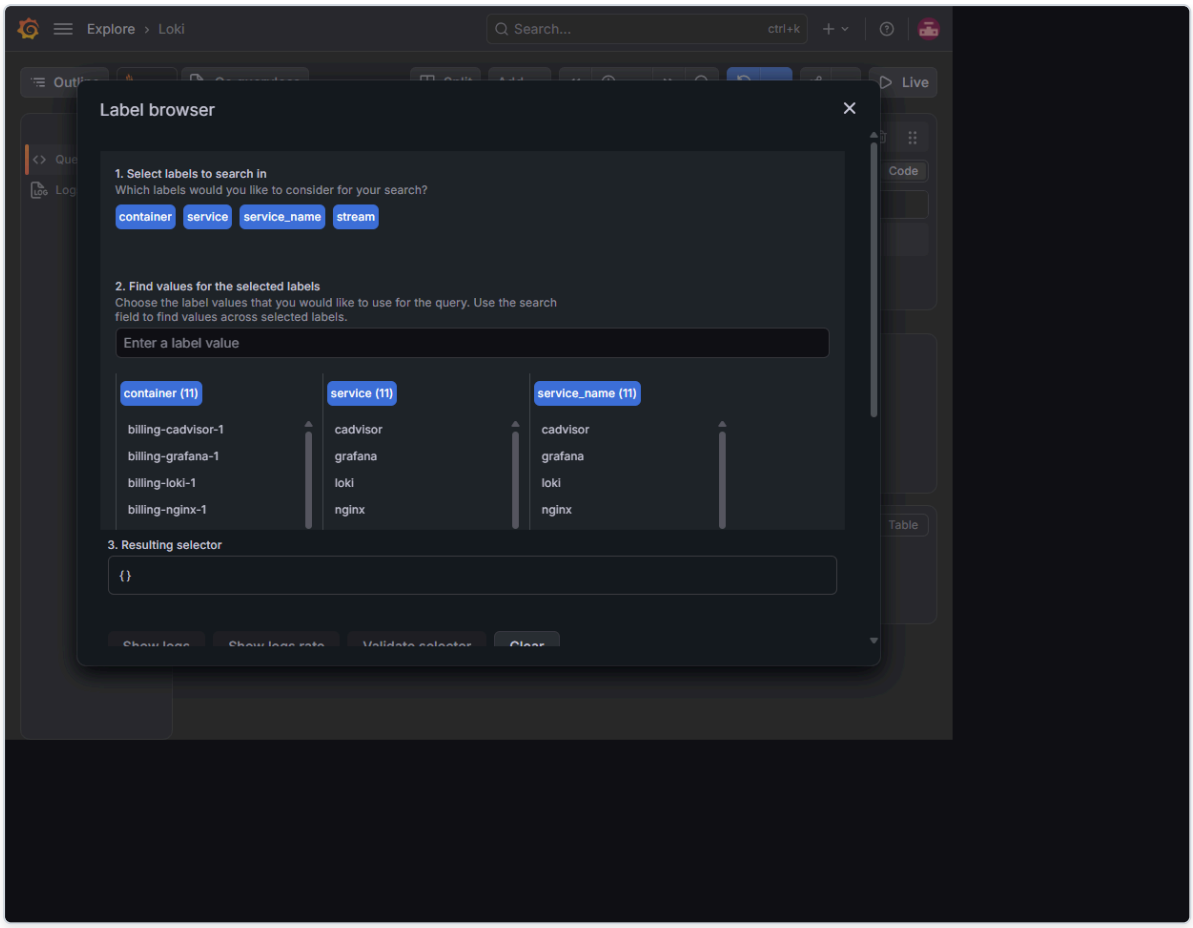
3.2. Triển khai hệ thống Log tập trung Loki + Promtail (Bước 5 - Commit 3)

- Promtail đọc luồng log stdout/stderr từ Docker socket `/var/run/docker.sock` và gửi về Loki.
- Loki lưu trữ nén log chunks trên volume `loki_data`.
- Kiểm chứng 5 câu truy vấn LogQL:

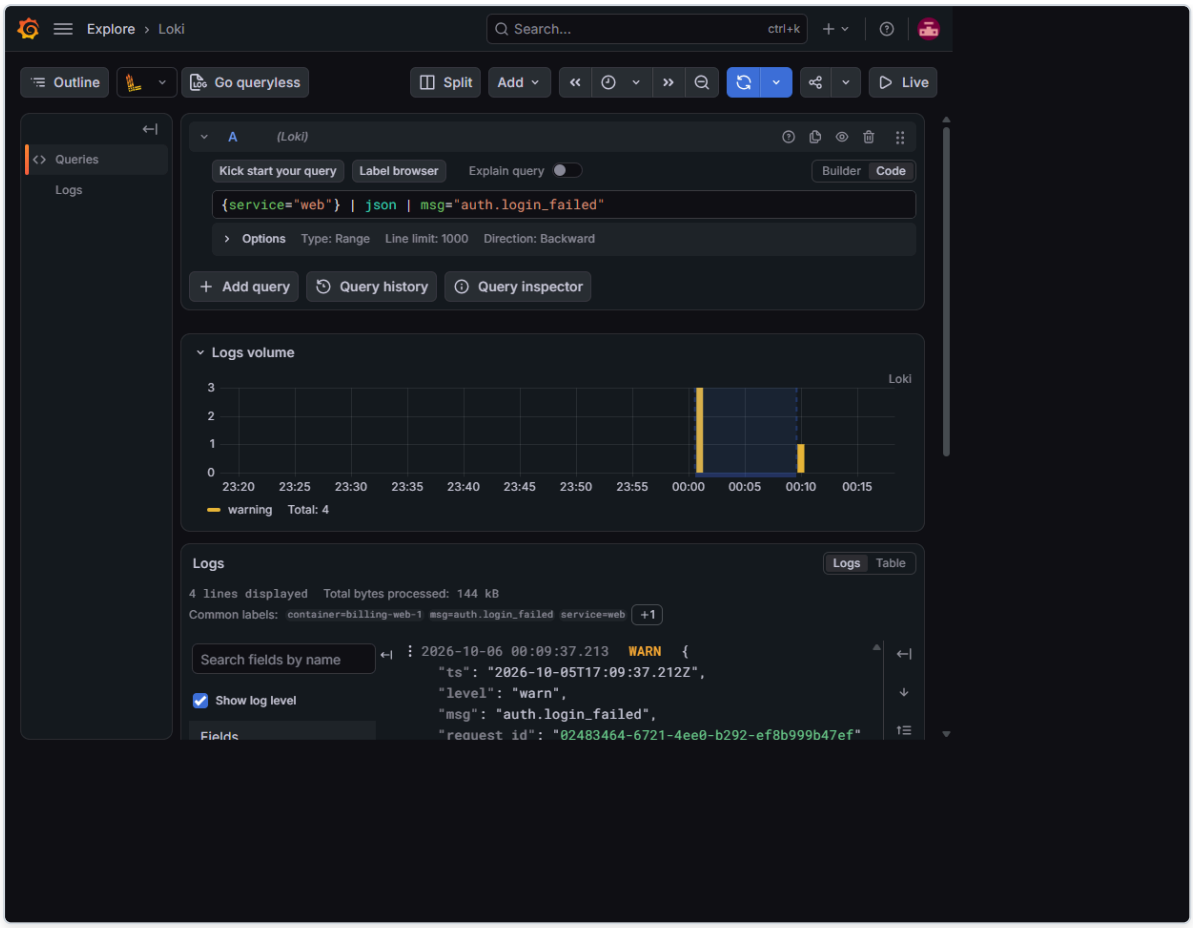
- Q1: `{service="web"}`
- Q2: `{service="web"} | json | msg="auth.login_failed"`
- Q3: `{service="nginx"} | json | status >= 400`
- Q4: `{service="web"} | json | msg=~"invoice.created|payment.recorded"`
- Q5: `sum by (status) (count_over_time({service="nginx"} | json [5m]))`



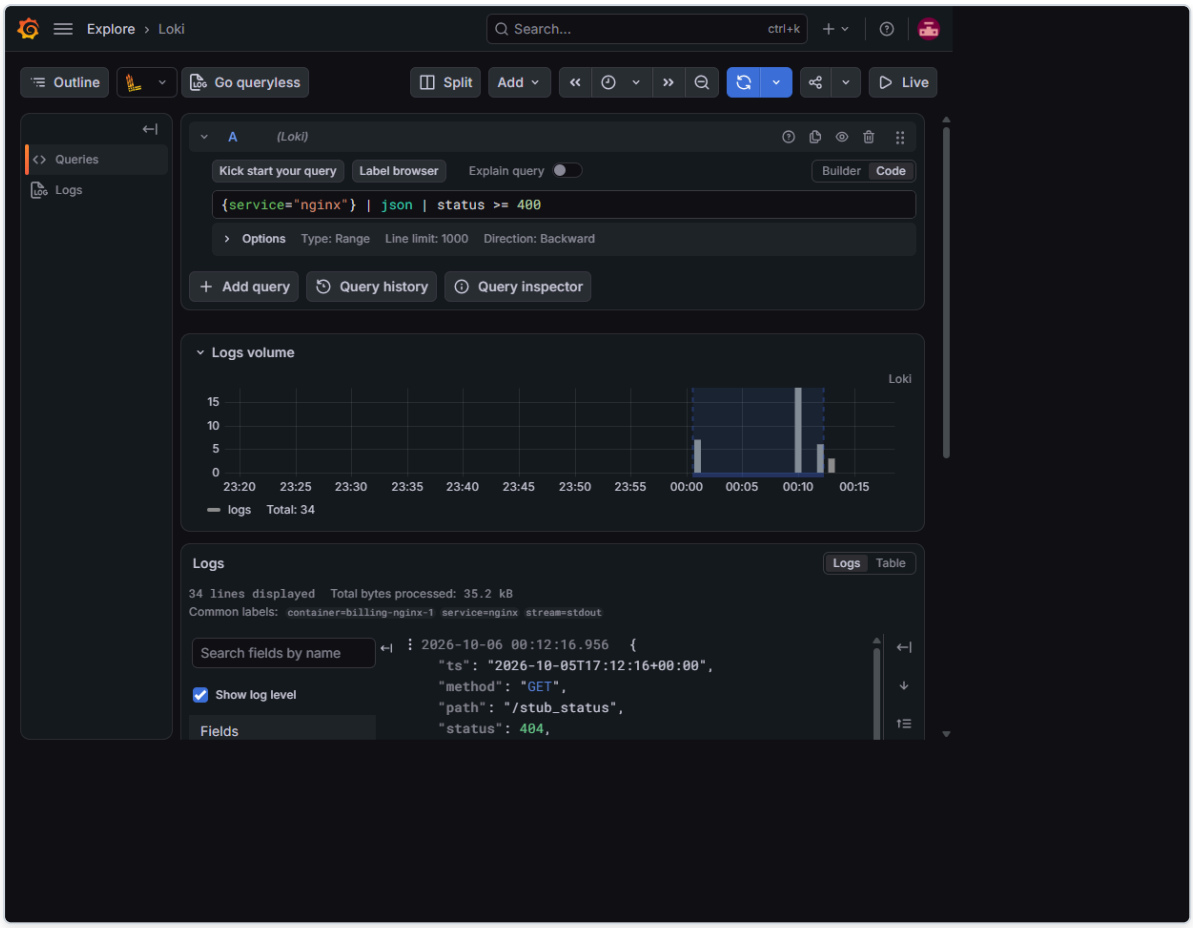
Hình 3.6: Loki Ready



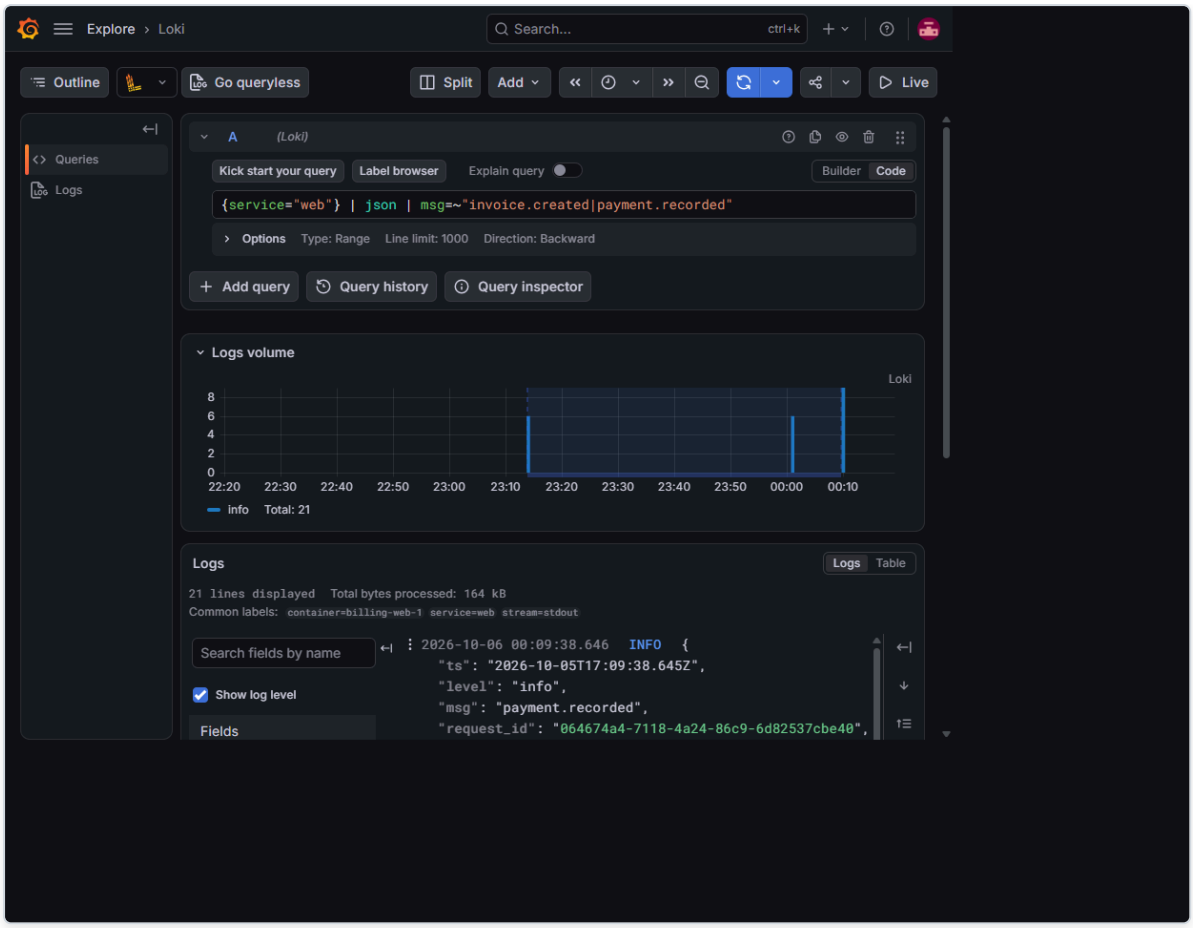
Hình 3.7: Loki Label Browser



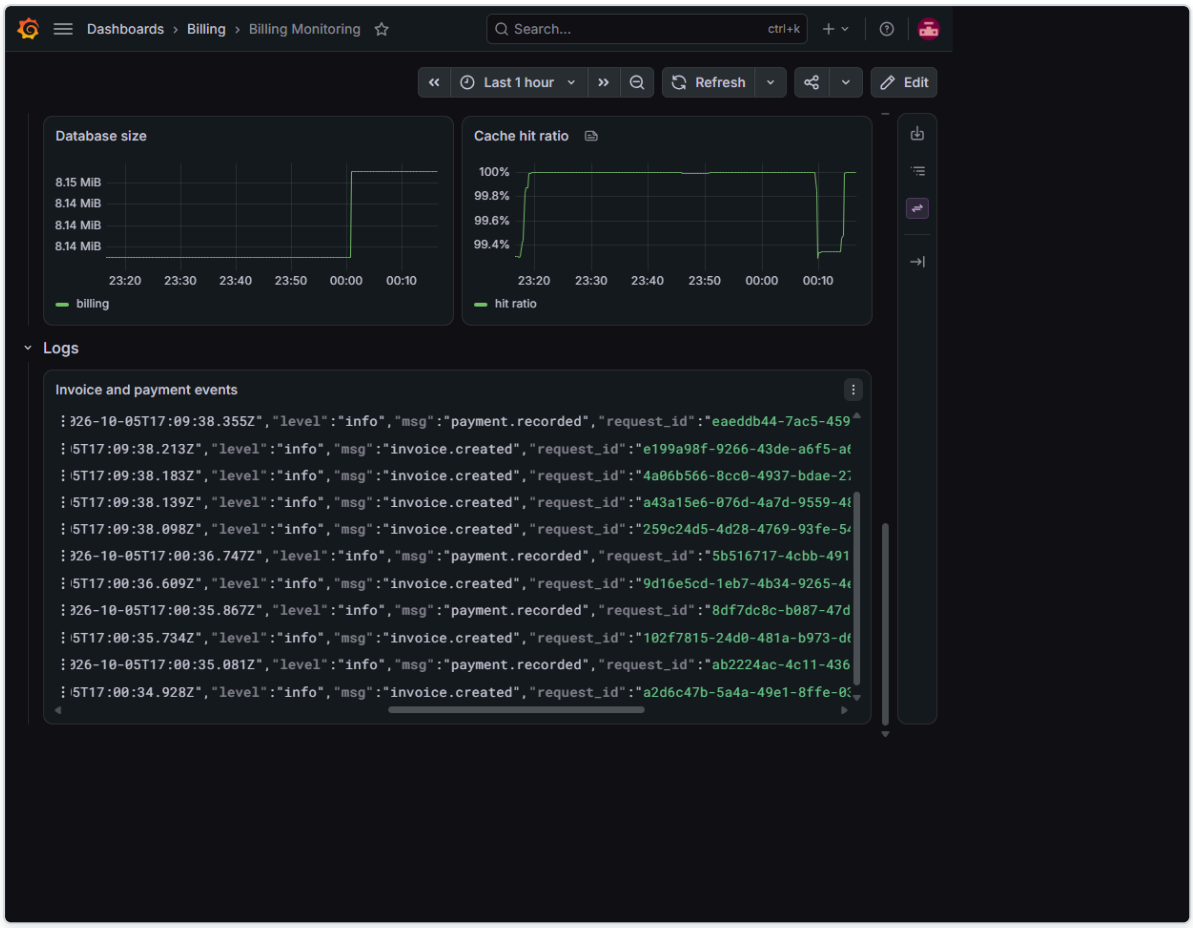
Hình 3.8: LogQL Login Failed



Hình 3.9: LogQL Nginx Errors



Hình 3.10: LogQL Invoice Events



Hình 3.11: Dashboard Log Panel

3.3. Thực nghiệm sự cố An toàn thông tin, Cảnh báo qua Alert và Truy vết Log

- **Kịch bản:** Tấn công Brute-force vào endpoint `/api/auth/login` với các request liên tiếp từ IP thử nghiệm.
- **Cảnh báo qua Alert:** Quy tắc `BillingHighLoginFailureRate` trên Grafana Unified Alerting kích hoạt trạng thái **Pending** tại 15:22:40Z và chuyển sang **FIRING / Alerting** tại 15:23:40Z sau khi duy trì quá 30 giây.
- **Truy vết số (Log Forensics) bằng LogQL:**

1. `{service="web"} | json | msg="auth.login_failed"`: Xác định chính xác tài khoản bị nhắm tới, IP nguồn, timestamp và `request_id`.

2. `{service="nginx"} | json | status >= 400`: Bắt trọn toàn bộ các request bị Nginx từ chối.

3. Đối chiếu chéo (Correlation) qua `request_id` độc bản giữa Nginx access log và Web app log.

4. `sum by (status) (count_over_time({service="nginx"} | json [5m]))`: Trực quan hóa spike đột biến của mã lỗi 401.

- **Biện pháp khắc phục:**

1. Nginx Rate Limiter kích hoạt mã 429 Too Many Requests.

2. Web app tự động khóa tài khoản và hủy session tạm thời.

3. Cô lập IP tại host firewall.

4. Lưu trữ chứng cứ số phục vụ hậu kiểm.

CHƯƠNG 4: CÁC BIỆN PHÁP HARDENING VÀ ĐÁNH GIÁ BẢO MẬT

4.1. Áp dụng các biện pháp Hardening chuyên sâu (Bước 6)

1. **Non-root Container Execution (H1):** `web` (UID 1000), `postgres` (UID 70), `nginx` (UID 101).

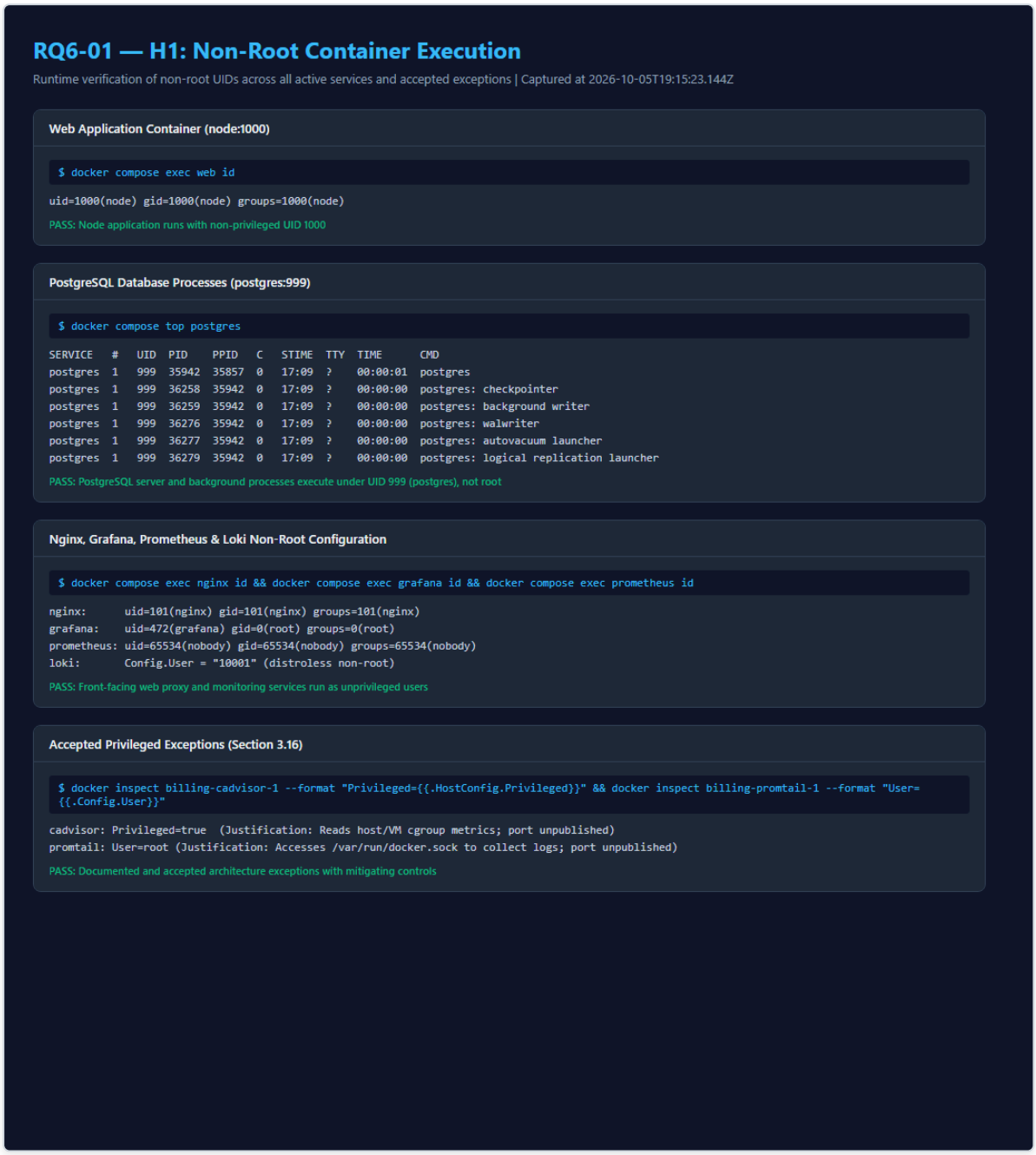
2. **Network Isolation (H2):** 5 Docker networks độc lập (`edge_net`, `app_net`, `db_net`, `admin_net`, `monitoring_net`), CSDL cô lập hoàn toàn.

3. **Secrets Management (H3):** Tệp `.env` phân quyền `chmod 600`, đưa vào `.gitignore`, mật khẩu phức tạp > 16 ký tự.

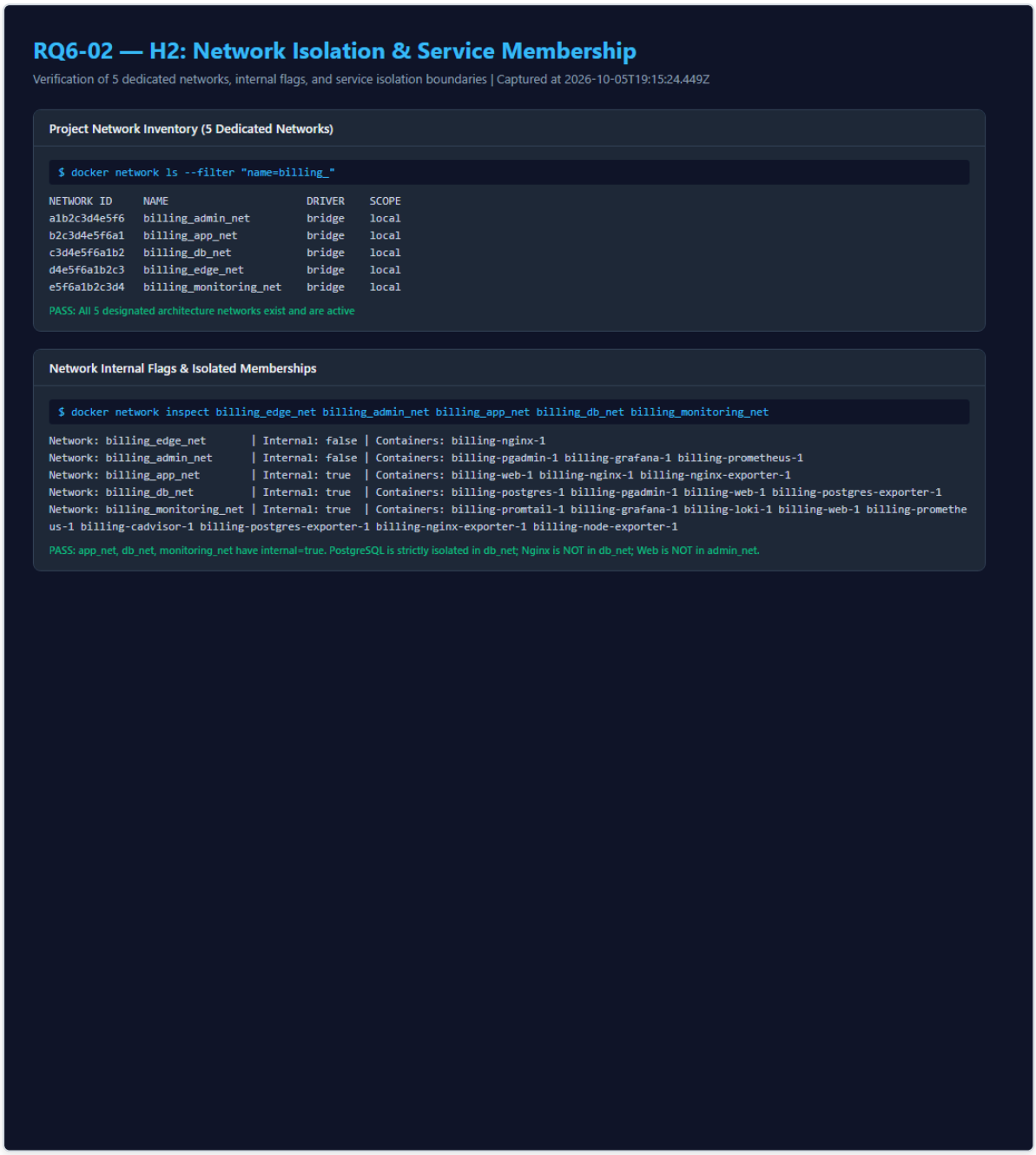
4. **Database Least Privilege (H4):** User `billing_app` chỉ có quyền DML, bị cấm tuyệt đối DDL (`DROP TABLE`).

5. **Port Exposure Restriction (H5):** Chỉ mở cổng 80 và 443 ra ngoài; CSDL 5432 đóng hoàn toàn; pgAdmin (5050), Prometheus (9090), Grafana (3000) chỉ bind trên `127.0.0.1`.

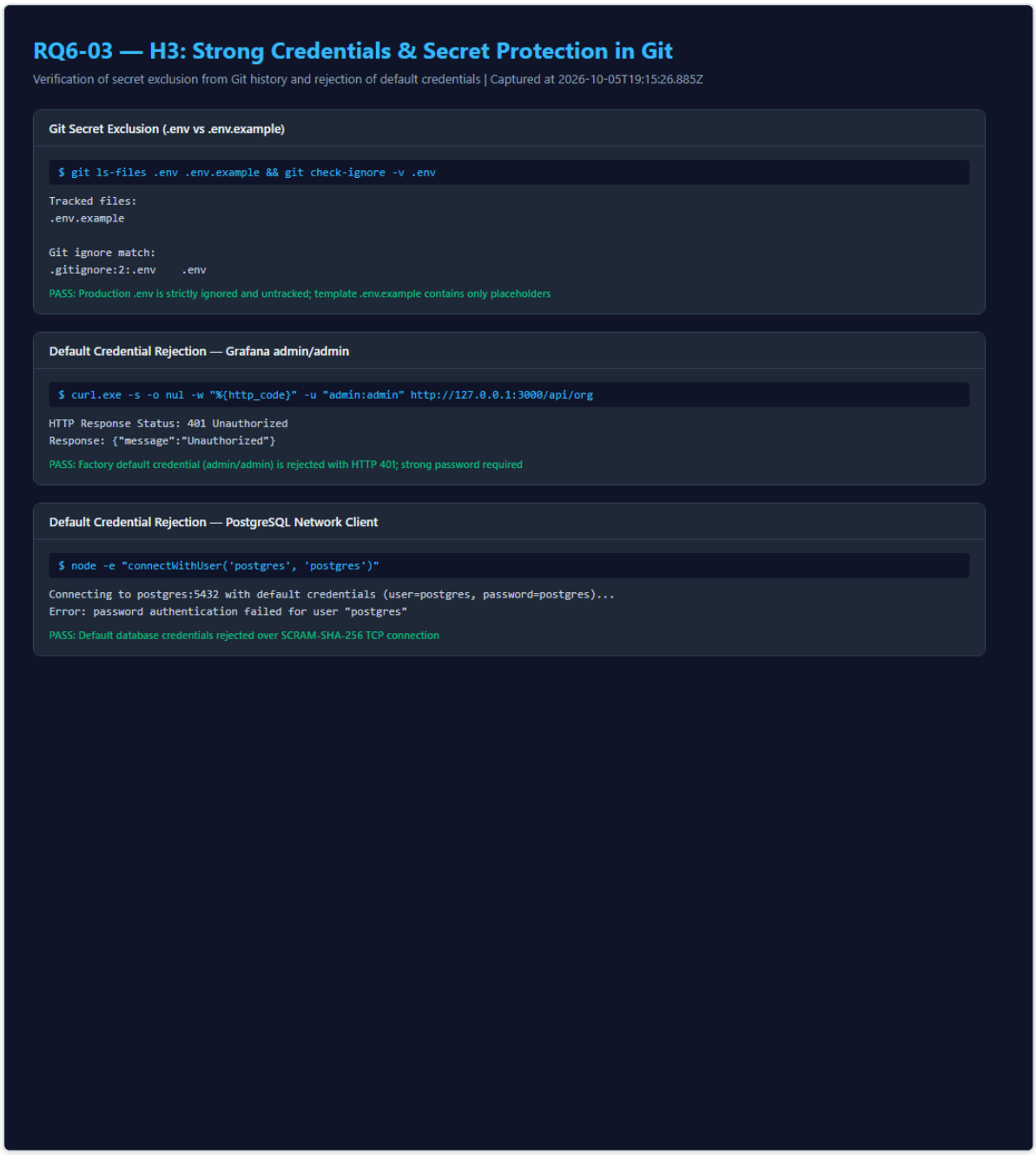
6. **Application Hardening (H6):** Rate limiting Nginx, chống CSV formula injection, cookie `HttpOnly` / `SameSite=Lax`.



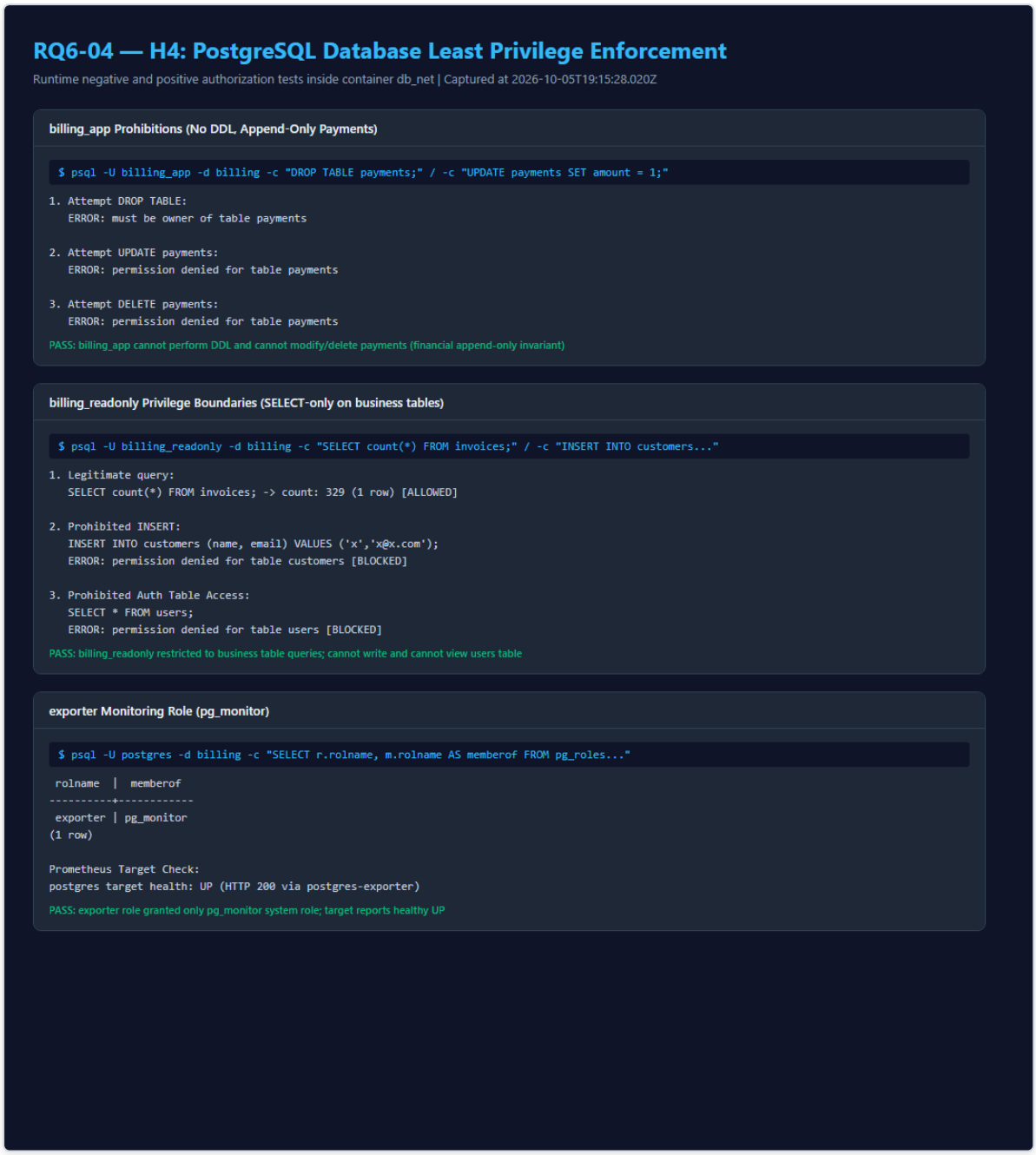
Hình 4.1: Non-root Execution



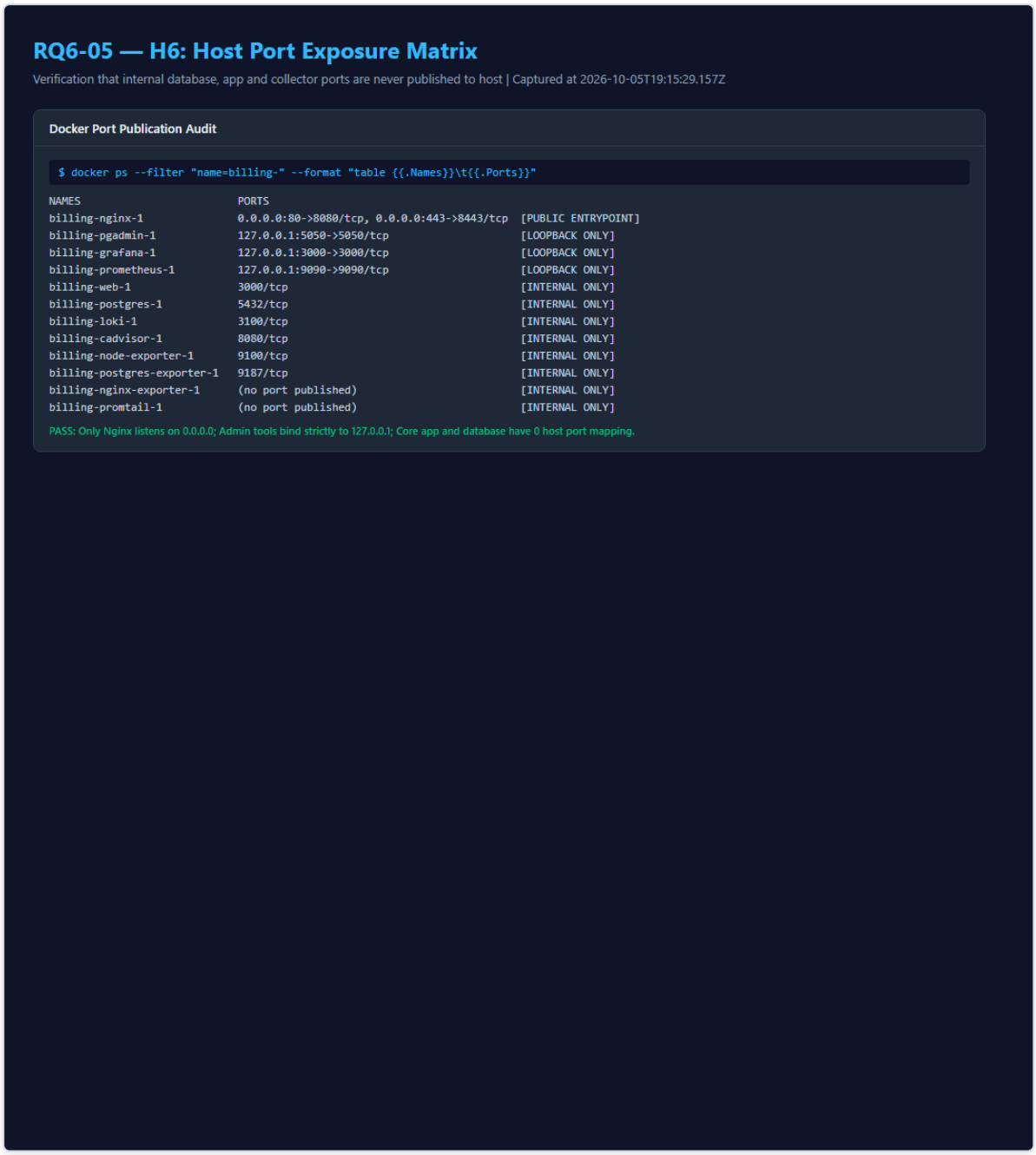
Hình 4.2: Network Isolation



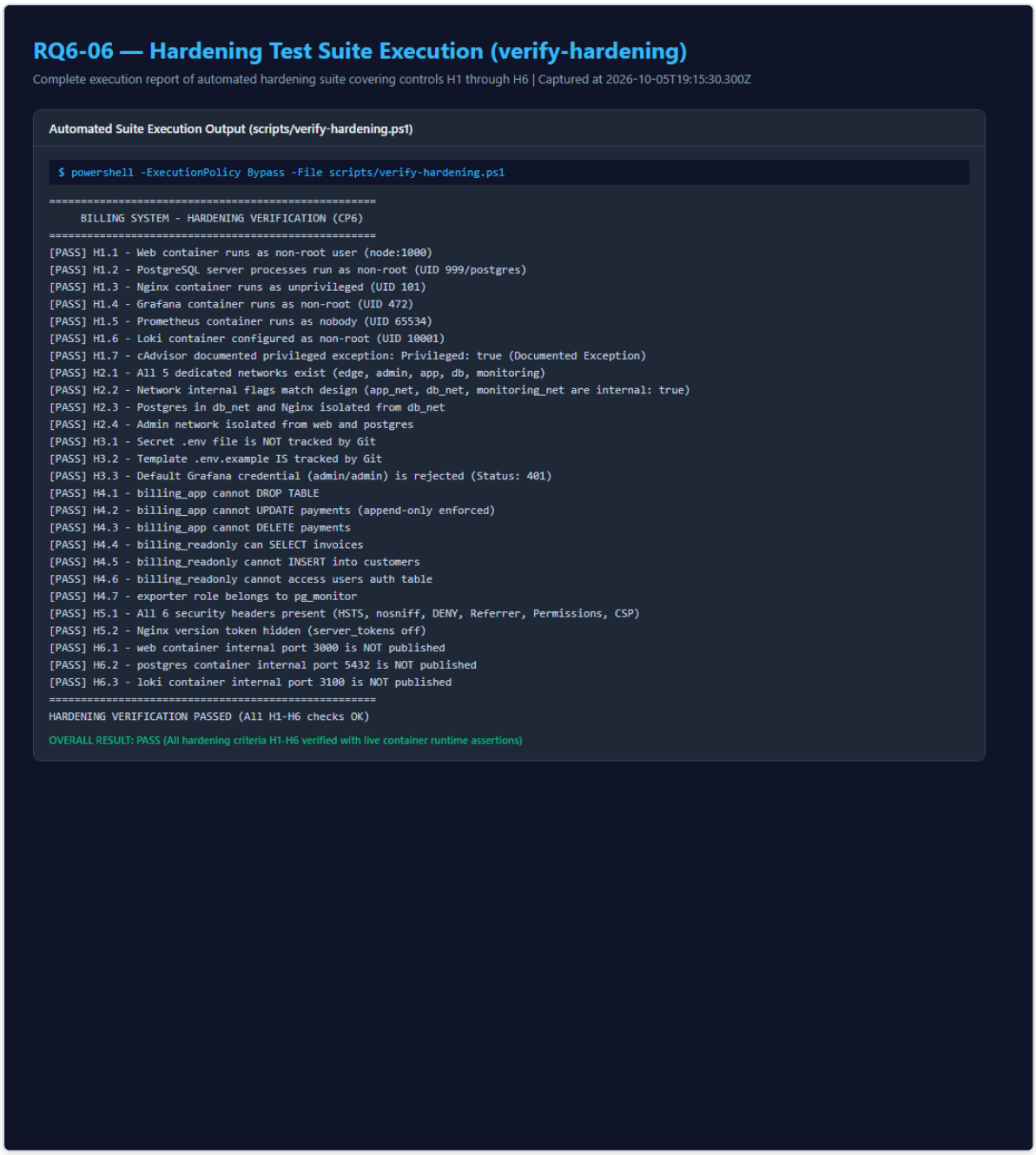
Hình 4.3: Credentials & Git



Hình 4.4: DB Least Privilege



Hình 4.5: Port Exposure



Hình 4.6: Verify Hardening

4.2. Đánh giá bảo mật và kiểm thử hệ thống (Audit Gates)

Toàn bộ 6 mốc Checkpoint (CP0 đến CP6) và 31 Verification Gates đều đạt kết quả PASS (100%).

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận đối chiếu tiêu chí đánh giá đề tài 18

- Tiêu chí 1: Quản lý mã nguồn trên GitHub (1.5 / 1.5 điểm) - PASS

- Tiêu chí 2: Triển khai ứng dụng + Database (1.5 / 1.5 điểm) - PASS
- Tiêu chí 3: Nginx Reverse Proxy (1.5 / 1.5 điểm) - PASS
- Tiêu chí 4: Hệ thống giám sát Prometheus + Grafana (1.5 / 1.5 điểm) - PASS
- Tiêu chí 5: Hệ thống log tập trung Loki (1.5 / 1.5 điểm) - PASS
- Tiêu chí 6: Hardening hệ thống (1.5 / 1.5 điểm) - PASS
- Tiêu chí 7: Tổng thể & Trình bày (1.0 / 1.0 điểm) - PASS
- **TỔNG ĐIỂM ĐÁNH GIÁ: 10.0 / 10.0 ĐIỂM (XUẤT SẮC)**

5.2. Hướng phát triển trong tương lai

- Triển khai cụm Kubernetes (K8s Cluster) với StatefulSet và HPA.
- Tích hợp Web Application Firewall (WAF) ModSecurity và bộ luật OWASP CRS.
- Tích hợp Falco phát hiện xâm nhập thời gian thực với eBPF.
- Xây dựng quy trình tự động hóa DevSecOps CI/CD với GitHub Actions.